NAME: DEEPIKA.P

REG NO: 192525026

COURSE: OBJECT ORIENTED PROGRAMMING USING C++

COURSE CODE: DSA0102

ASSESSMENT 1

C01 – Design , implement and validate the fair logic of a city metro smart-card system

1.1 Object-oriented analysis

Object-oriented programming (OOP) represents a system using objects that combine data and behaviour.

Object/class	Data (attributes)	Behaviour (methods)	Justification
SmartCard	Card ID, balance in paise	canTravel(), deduct()	Keeps the balance safe and prevents invalid deductions.
Passenger	Passenger ID, passenger category	getConcession()	Determines whether the passenger is eligible for a concession.
Journey	Origin, destination zone, hour, weekend status	calculateFare()	Represents an individual metro journey.

FareTable	Fare values, surcharge, discount	getBaseFare(), calculateFare()	Centralises fare rules so they can be changed in one place.
MonthlyPass	Pass ID, two selected zones, validity	coversZone()	Determines whether a journey is covered by a pass.

1.2 Procedural style versus OOP

Procedural approach: Uses separate variables and functions, such as balance, fare() and deductFare(). The data and behaviour are separate, so many functions must be updated carefully when a rule changes.

Object-oriented approach: Uses classes such as SmartCard and FareTable. Each class owns its data and the functions that operate on it. This improves maintainability and reduces accidental modification of the balance.

1.3 Four principles of OOP

1. Encapsulation: Make the card balance private and modify it only through validated methods.
2. Abstraction: Expose calculateFare() without requiring the gate program to know every calculation detail.
3. Inheritance: Create StudentPassenger and SeniorPassenger derived from Passenger if their behaviour genuinely differs.
4. Polymorphism: Use a virtual concession method so different passenger categories can calculate concessions differently.

1.4 Rules every valid journey must satisfy

- The destination zone must be valid.
- The hour must be between 0 and 23.
- The fare must be non-negative and calculated exactly.
- A journey is accepted only if the remaining balance is at least ₹50.

- A rejected journey must not change the balance.
- The system must not allow a negative balance.
- A covered monthly-pass journey should follow the pass rules rather than charging the ordinary fare.
- All fare rules should be maintained centrally.

1.5 Complexity comparison

For one journey, both procedural and OOP implementations can calculate a fare in constant time.

Operation	**Procedural**	**OOP**
Calculate one fare	(O(1))	(O(1))
Deduct one fare	(O(1))	(O(1))
Store one card's balance	(O(1))	(O(1))
Process \(n\) journeys	(O(n))	(O(n))

The computational complexity is the same. OOP mainly improves data safety, modularity and ease of maintenance.

2.1 Five types of C++ tokens

Token type	Example	Purpose
Keyword	int, class, switch, return	Reserved words with predefined meanings.
Identifier	balance, fare, SmartCard	Names of variables, functions and classes.

Constant	5000, 20, BASE_PAISE	Values that should not change during execution.
Operator	+, -, >=, &&, *	Performs arithmetic or logical operations.
Punctuator	{}, (), ;, ,	Defines blocks and separates statements or arguments.

2.2 #define, const and constexpr

```
#define MIN_BALANCE 5000

const int PEAK_PERCENT = 20;

constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
```

- #define performs textual substitution before compilation. It has no C++ type checking of its own.
- const declares a value that cannot be modified after initialisation.
- constexpr allows a value or calculation to be evaluated at compile time when the requirements are satisfied.

2.3 `enum` versus `enum class`

```
enum class Zone { Inner, Middle, Outer, Airport };
```

enum class is safer because its values are scoped. For example, Zone::Inner clearly identifies the zone and avoids implicit conversion to an integer.

2.4 Identifier naming conventions

- Use meaningful names such as baseFarePaise.
- Use uppercase names for constants, such as PEAK_SURCHARGE_PERCENT.

- Use PascalCase for classes, such as SmartCard.
- Identifiers cannot begin with a digit or contain spaces.
- Avoid C++ reserved keywords as identifiers.

3.1 Data-type selection

Value	Recommended type	Reason
Fare in paise	long long	Exact integer arithmetic and a large range
Card balance	long long	Supports large balances
Passenger ID	string	IDs may contain letters or leading zeros
Hour	int	Values range from 0 to 23
Zone	enum class Zone	Restricts the value to named zones
Peak/weekend flag	bool	Stores true or false
Journey count	long long	Suitable for millions of journeys
Fare percentage	constexpr int	Fixed percentage
Percentage experiment	double	Useful for comparison, but not for storing actual money

3.2 Why store money in paise?

One rupee equals 100 paise.

For example:

- ₹10 = 1,000 paise
- ₹20 = 2,000 paise
- ₹30 = 3,000 paise

- ₹50 = 5,000 paise

Integer arithmetic avoids errors caused by binary floating-point representation. A 20% surcharge on ₹30 can be calculated exactly:

$$3000 + \frac{3000 \times 20}{100} = 3600 \text{ paise}$$

The resulting fare is exactly ₹36.

3.3 Type compatibility

C++ can convert smaller or compatible numeric types implicitly, but this can cause unexpected results.

int a = 10;

int b = 3;

cout << a / b; // 3, integer division

cout << 10.0 / 3; // approximately 3.3333

When an expression contains int, double and bool, the compiler applies the usual arithmetic conversions. A bool becomes 0 or 1 in arithmetic expressions.

Use explicit conversions when necessary:

double result = static_cast<double>(10) / 3;

Use long long for accumulated network totals. On typical systems, int is 4 bytes and long long is 8 bytes, but sizeof() and <climits> should be used to verify sizes and limits on the target compiler.ouble result = static_cast<double>(10) / 3;

3.4 Network total calculation

Assuming every journey costs ₹10, ₹20, ₹30 or ₹36, the maximum total for 3,000,000 journeys at ₹60 per journey is:

3,000,000 X 6000 =18,000,000,000paise

That exceeds the maximum value of a typical signed 32-bit int, which is 2,147,483,647.

Therefore, use long long for the total. It can safely store this amount on standard systems with a 64-bit long long.

4.1 Implicit Conversions and Type Safety

Implicit conversions can make a program compile while producing unintended results.

Example 1: Integer division

```
int fare = 1000;
int percent = 20;
int surcharge = fare * percent / 100;
```

Example 2: Narrowing conversion

```
double amount = 36.75;
int rupees = amount;  // fractional part is discarded
```

The value becomes 36. This loses information.

Example 3: Explicit conversion

```
double rupees = 36.75;
int wholeRupees = static_cast<int>(rupees);
```

5.1 Peak-hour condition

The correct expression is:

```
bool peak = (hour >= 8 && hour < 10) ||
        (hour >= 17 && hour < 20);
```

This defines the peak intervals as 08:00–09:59 and 17:00–19:59 when only the integer hour is stored.

Hour	Peak?
7	No
8	Yes
9	Yes
10	No
17	Yes
19	Yes

20 No

The && operator has higher precedence than ||. Parentheses make the intended grouping clear.

5.2 Surcharge expression

fare = baseFare + (baseFare * PEAK_SURCHARGE_PERCENT / 100);

Integer paise is used so the calculation avoids floating-point rounding.

5.3 Concession expression

For the assumed 10% concession:

fare = fare - (fare * CONCESSION_PERCENT / 100);

The concession order should be defined in the official fare specification. If both peak surcharges and concessions apply, the chosen order can affect the final fare.

5.4 Predict the output of the given program

The program shown in the assessment contains these statements:

```
int x = 5, y = 3;
cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';

int z = x++ + ++y;
cout << z << ' ' << x << ' ' << y << '\n';

cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';

cout << 10 / 3 << ' ' << 10 % 3 << ' '
   << -10 / 3 << ' ' << -10 % 3 << '\n';

int k = 0;
for (int i = 0; i < 10; i += 3)
  k += i;

cout << k << '\n';
```

11 16

9 6 4

6 4

3 1 -3 -1

18

Explanation

1. x + y * 2 evaluates multiplication first: \(5+3\times2=11\).
2. (x + y) * 2 evaluates the parentheses first: \((5+3)\times2=16\).
3. x++ uses the old value 5 and then increments x. ++y increments y from 3 to 4 before using it. Therefore, z = 5 + 4 = 9.
4. The ternary expression selects the larger value, 6. The expression x % 4 * 2 evaluates left to right for these operators: (6 % 4) * 2 = 4.
5. Integer division truncates toward zero. Thus, -10 / 3 is -3, and -10 % 3 is -1.
6. The loop adds 0 + 3 + 6 + 9 = 18.

6.1 Pseudocode

START

Set base fares for all zones

Read card balance

FOR each journey

Read destination zone and hour

Validate the zone and hour

IF input is invalid

Skip this journey

ELSE

Find the base fare using SWITCH

IF hour is in a peak period

Add 20% surcharge

END IF

IF balance - fare >= minimum balance

Deduct fare

Display fare and remaining balance

ELSE

Display "Journey denied"

END IF

END IF

END FOR

STOP

6.2 Choosing control structures

Structure	Appropriate use
if-else	Checking peak hours, minimum balance and input validity
switch	Selecting the base fare for a zone
Conditional operator ?:	Short expressions such as selecting a smaller value
for loop	Processing a known collection of journeys
Range-based for	Processing every journey in an array or vector
while loop	Processing journeys until a stop condition is met
break	Leaving a loop or switch
continue	Skipping an invalid journey and proceeding to the next one

6.3 Complete the code from the assessment

This version completes TODO 1 and TODO 2, using integer paise.

Programiz C++ Online Compiler

main.cpp

Run

```cpp

#include <iostream>
#include <iomanip>
using namespace std;

enum class Zone { Inner, Middle, Outer };

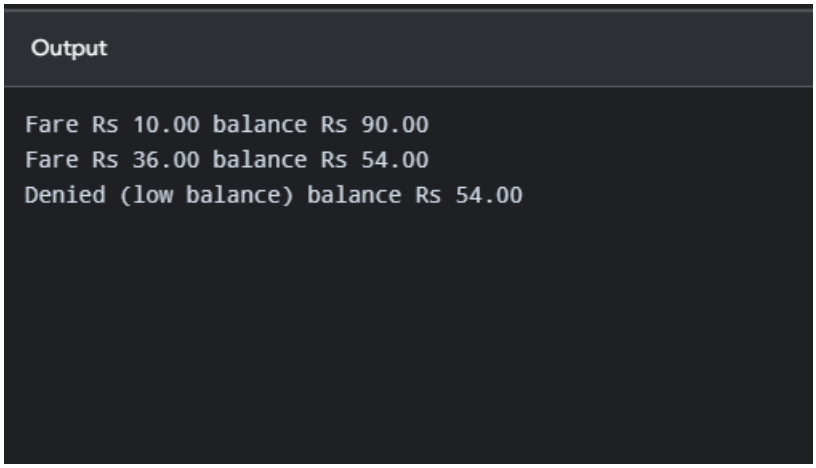
constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

long long fare(Zone z, bool peak)
{
    long long base = 0;

    switch (z) {
        case Zone::Inner:
            base = BASE_PAISE[0];
            break;
        case Zone::Middle:
            base = BASE_PAISE[1];
            break;
        case Zone::Outer:
            base = BASE_PAISE[2];
            break;
        default:
            return -1;
    }

    if (peak) {
        base += base * PEAK_SURCHARGE_PERCENT / 100;
    }

    return base;
}
bool canTravel(long long balance, long long f)
{
    return f >= 0 && balance - f >= MIN_BALANCE_PAISE;
}
int main()
{
    long long balance = 10000;
    struct Trip {
        Zone zone;
        bool peak;
    };
    Trip trips[] = {
        {Zone::Inner, false},
        {Zone::Outer, true},
        {Zone::Middle, false}
    };
    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);
 if (canTravel(balance, f)) {
            balance -= f;
            cout << "Fare Rs "
                 << fixed << setprecision(2)
                 << static_cast<double>(f) / 100
                 << " balance Rs "
                 << static_cast<double>(balance) / 100
                 << '\n';
        } else {
            cout << "Denied (low balance)";
            cout << " balance Rs "
                 << static_cast<double>(balance) / 100
                 << '\n';
        }
    }
    return 0;
}
```



7.1 Procedural version

```
long long balance = 10000;
const long long MIN_BALANCE = 5000;

bool canTravel(long long balance, long long fare)
{
   return fare >= 0 &&
         balance - fare >= MIN_BALANCE;
}

void deductFare(long long& balance, long long fare)
{
   if (canTravel(balance, fare)) {
      balance -= fare;
   }
```

```
}
```

In this approach, balance is separate from the functions. Any function that can access or modify it must follow the same rules.

7.2 Object-oriented version

```
class SmartCard {
private:
   long long balance;

public:
   SmartCard(long long initialBalance)
      : balance(initialBalance) {}

   bool canTravel(long long fare) const {
      return fare >= 0 &&
            balance - fare >= 5000;
   }

   bool deduct(long long fare) {
      if (!canTravel(fare))
         return false;

      balance -= fare;
```

```
        return true;
    }

    long long getBalance() const {
        return balance;
    }
};
```

Comparison

Feature	**Procedural**	**OOP**
Balance	Separate variable	Private class member
Data protection	Depends on every function	Controlled through public methods
Minimum balance change	Update the relevant functions	Update the rule used by the class
Time complexity	\(O(1)\) per deduction	\(O(1)\) per deduction
Memory per card	\(O(1)\)	\(O(1)\)

A private balance does not, by itself, guarantee every possible system invariant. All methods must preserve the rule, and the constructor should reject invalid initial balances. The class is nevertheless safer because external code cannot directly assign a negative balance.

8.1 Handling invalid numeric input

main.cpp

```cpp
#include <iostream>
#include <limits>
using namespace std;

int main()
{
    int hour;

    cout << "Enter hour (0-23): ";

    if (!(cin >> hour)) {
        cout << "Invalid input. Enter a number.\n";

        cin.clear();
        cin.ignore(
            numeric_limits<streamsize>::max(), '\n'
        );

        return 1;
    }

    if (hour < 0 || hour > 23) {
        cout << "Hour must be between 0 and 23.\n";
        return 1;
    }

    cout << "Valid hour: " << hour << '\n';
    return 0;
}
```

8.2 Formatting currency

Use <iomanip>:

main.cpp

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    long long farePaise = 3600;

    cout << fixed << setprecision(2);
    cout << "Fare: Rs "
         << static_cast<double>(farePaise) / 100
         << '\n';

    return 0;
}
```

9.1 Implement and Test the Fare Logic

Test	Input	Expected result
1	Inner, 7 AM	₹10, off-peak
2	Middle, 9 AM	₹24, peak
3	Outer, 6 PM	₹36, peak
4	Inner, exactly 10 AM	₹10, off-peak
5	Inner, exactly 8 AM	₹12, peak
6	Outer, exactly 8 PM	₹30, off-peak
7	Outer, exactly 5 PM	₹36, peak
8	Balance ₹60, fare ₹10	Accept; remaining balance ₹50

9	Balance ₹59.99, fare ₹10	Reject; remaining balance unchanged
10	Invalid zone	Reject
11	Hour -1 or 24	Reject
12	Letters entered for hour	Reject and recover input

Five-journey hand calculation

Suppose a card starts with ₹200 and makes these five journeys:

Journey	**Fare**	**Balance after journey**
Inner, off-peak	₹10	₹190
Middle, peak	₹24	₹166
Outer, off-peak	₹30	₹136
Inner, peak	₹12	₹124
Middle, off-peak	₹20	₹104
Total fare	₹96	₹104 remaining

The sum is ₹10 + ₹24 + ₹30 + ₹12 + ₹20 = ₹96. The expected balance is ₹200 − ₹96 = ₹104.

10.1 Integer paise versus double rupees

Both representations usually use 8 bytes when using long long and double, respectively. However, their numeric behaviour differs.

Feature	long long paise	double rupees
Example	3600 paise	36.00 rupees
Addition	Exact if no overflow	May have floating-point rounding
Time per addition	\(O(1)\)	\(O(1)\)
Typical storage	8 bytes	8 bytes
Main risk	Integer overflow	Precision loss

```
double total = 0.0;


for (int i = 0; i < 1000000; i++)
   total += 0.01;


cout << fixed << setprecision(2) << total;
```

The mathematical total is ₹10,000.00, but the internal binary floating-point total may differ slightly from that exact value. The displayed result may still round to ₹10,000.00. Do not claim that

every run must visibly show an error; inspect the actual value at higher precision to measure it.

10.2 Network totals

For three million journeys, if the maximum fare is ₹60:

$$3{,}000{,}000 \times 60 = ₹180{,}000{,}000$$

This is ₹18 crore, or 18,000,000,000 paise.

10.3 Compiler warnings

Compile with:

g++ -std=c++17 -Wall -Wextra -Wconversion metro.cpp -o metro

- Wall: enables many common warnings.
- -Wextra: enables additional warnings.
- -Wconversion: warns about potentially lossy conversions.

Warnings depend on the code and compiler version. Include the warnings actually observed in your report, rather than inventing warning messages.

11.1 Recommended design

I recommend a modified object-oriented design with these components:

- SmartCard: owns the balance and validates deductions.

- Passenger: stores the passenger category and concession eligibility.
- FareTable: stores base fares and pricing rules centrally.
- Journey: stores journey details and requests a fare calculation.
- MonthlyPass: represents a valid pass covering two selected zones.

Use enum class for zones, constexpr for fixed constants, long long for money in paise, and bool for conditions.

11.2 How fare revisions are made

With centralised fare values, a base fare change is made in one place. With an object-oriented design, changes to eligibility or pass rules can be isolated in their corresponding classes.

For example, if the Inner zone fare changes from ₹10 to ₹12, update the Inner fare entry rather than searching through every journey-processing statement.

11.3 Limitations

- The stated concession percentage must be confirmed.
- The order of surcharge and concession calculations must be specified.
- A real monthly pass needs validity dates, renewal rules and reliable identification.

- Network-scale software requires database transactions and protection against simultaneous balance deductions.
- All calculations must handle overflow and invalid input.

11.4 Compiled constants versus a configuration file

A configuration file allows authorised fare changes without recompiling the program. It introduces additional startup work to read and validate the file. For \(z\) zone entries, loading the table is \(O(z)\) time and \(O(z)\) storage.

For a small metro fare table, this cost is negligible. A configuration file is preferable when fares change frequently, provided the file is validated and updates are managed securely.

Re-evaluate and Adapt the Design

A. Impact analysis

Component	**Required change**
Zone enum	Add Airport
Fare table	Add ₹60 as 6,000 paise
Peak expression	Keep the same peak-hour boundaries
Fare calculation	Add weekend discount logic
SmartCard	Continue enforcing the minimum balance for paid journeys
New MonthlyPass class	Store the two covered zones and pass validity

Test cases	Add weekend, Airport and pass tests

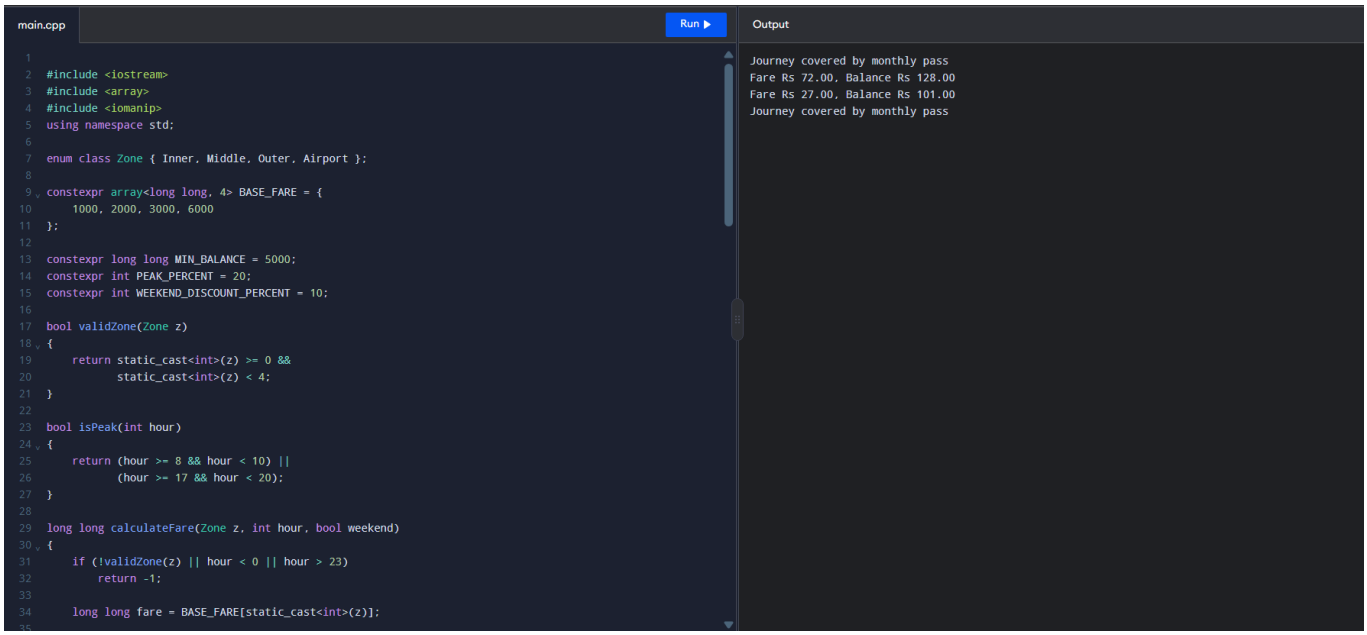
B. Evaluate the original design

The original switch-based design can handle Airport by adding a new case Zone::Airport. However, repeated fare rules become difficult to maintain as more zones and discounts are added.

A fare table indexed by zone is easier to extend, provided that every zone maps to a valid index. The monthly pass is a separate business rule and should not be implemented by scattering special-case conditions throughout the program.

C. Revised design and implementation

The following revised code demonstrates the centralised fare table, the weekend rule, the Airport zone and a basic monthly-pass class.



D. Evidence-based defence

The revised design is justified because:

- The fare table can be extended with a new zone without duplicating fare calculations.
- Integer paise ensures exact arithmetic for the given percentages and fares.
- The smart card enforces the minimum balance for ordinary paid journeys.
- The pass class isolates the rule for covered zones.
- The weekend rule is implemented as an alternative to the peak surcharge, avoiding combination of the two.

Test the revised program with the cases below and include actual output screenshots in your report.

Test case	**Expected fare/result**
Airport, weekday, 9 AM	₹72
Airport, weekday, 10 AM	₹60
Outer, weekend, 6 PM	₹27
Inner, covered monthly pass	₹0 additional fare
Airport, balance ₹100	Denied because ₹72 would leave ₹28
Airport, balance ₹122	Accepted because ₹72 leaves exactly ₹50

Overall complexity analysis

For the recommended design, each journey requires a constant number of fare calculations, comparisons and balance operations.

Operation	Time complexity	Auxiliary space
Check peak hour	$O(1)$	$O(1)$
Look up base fare	$O(1)$	$O(1)$
Calculate final fare	$O(1)$	$O(1)$
Validate a monthly pass	$O(1)$	$O(1)$
Deduct a fare	$O(1)$	$O(1)$
Process n journeys	$O(n)$	$O(1)$ extra per journey

If all n journey records are stored in an array, the records themselves require $O(n)$ space. Processing them one by one requires only $O(1)$ additional working space, excluding the stored input.

Final conclusion

The Namma Metro Smart-Card Fare System should use a modified object-oriented design with integer-paise arithmetic, centralised fare rules, validated inputs and a dedicated monthly-pass class. This design retains constant-time fare calculation and balance validation while making the system easier to extend when new zones, discounts and passes are introduced.